

# **Campus Navigator: Backend System and Web-Based Administration Client**

By

Pui Hock Yang



FACULTY OF COMPUTING AND  
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF  
MANAGEMENT AND TECHNOLOGY  
KUALA LUMPUR

ACADEMIC YEAR  
2025/26

# **Campus Navigator: Backend System and Web-Based Administration Client**

By

Pui Hock Yang

Supervisor: Prof. Ts. Dr. Tew Yiqi

A project report submitted to the  
Faculty of Computing and Information Technology  
in partial fulfillment of the requirement for the  
Software Engineering

Faculty of Computing and Information Technology  
Tunku Abdul Rahman University of Management and Technology  
Kuala Lumpur

**Copyright by Tunku Abdul Rahman University of Management and Technology.**

All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

# Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

---

Pui Hock Yang

Software Engineering

ID: 24WMR01790

## Abstract

Campus Navigator requires a management view of indoor navigation activity beyond the mobile user's own route. This project develops the backend integration and web-based administration client that expose navigation summaries and named user positions on a shared two-dimensional campus map. The system separates current presence from historical analytics through three Go services: a Presence Gateway, a Trajectory Worker and an Analytics API. Redis maintains active sessions and buffers events, while ClickHouse stores trajectory and journey records for aggregation. The administration client provides two pages. A dashboard summarises navigation starts, journey outcomes and popular destinations for the current day or week using readable place names. A live map displays active users with names and supports limits of 10, 20, 30 or all available markers. An iterative implementation process addressed map-origin alignment, end-to-end display-name propagation and the distinction between representative mobile snapshots and complete administrative snapshots. Local Docker verification used 100 simulated sessions sending positions through the backend rather than generating markers in the browser. The verification demonstrated named moving users and reconciled dashboard counts under that synthetic workload. Deployment scripts and documentation prepare the services and frontend for a Google Compute Engine virtual machine with public HTTPS and private backend HTTP. Public cloud deployment, real-world positioning accuracy and production-scale capacity remain unverified. The resulting prototype provides a focused operational view while retaining clear boundaries around privacy, data durability and future access control.

## Acknowledgement

I acknowledge Prof. Ts. Dr. Tew Yiqi as the supervisor of this project and thank the Faculty of Computing and Information Technology at Tunku Abdul Rahman University of Management and Technology for the academic setting in which this work is presented. I also acknowledge the maintainers of the open-source tools and the shared Campus Navigator resources used by this implementation. Their contributions are identified where relevant in this report.

# Table of Contents

## Abstract

## Acknowledgement

### 1 Introduction

- 1.1 Project Objectives
- 1.2 Project Background
- 1.3 Advantages and Contributions.....
- 1.4 Project Plan.....
- 1.5 Project Team and Organisation.....
- 1.6 Chapter Summary and Evaluation.....

### 2 Literature Review.....

- 2.1 Live Communication Approaches.....
- 2.2 Transient Presence and Event Processing.....
- 2.3 Historical Aggregation and Data Meaning
- 2.4 Deployment and Transport Options.....
- 2.5 Comparison and Project Justification.....
- 2.6 Chapter Summary and Evaluation.....

### 3 Methodology and Requirements Analysis.....

- 3.1 Development Methodology.....
- 3.2 Requirements Gathering.....
- 3.3 Stakeholders and Use Cases.....
- 3.4 Functional and Nonfunctional Requirements
- 3.5 Chapter Summary and Evaluation.....

### 4 System Design.....

- 4.1 System Context.....
- 4.2 User Interface Design.....
- 4.3 Data Design
- 4.4 Dashboard Semantics.....
- 4.5 Security and Privacy Design.....
- 4.6 Presence and Analytics Processes.....
- 4.7 Software Architecture and Interfaces.....
- 4.8 Coordinate and Display Algorithms
- 4.9 Chapter Summary and Evaluation.....

### 5 Implementation and Testing.....

- 5.1 Implementation Structure.....
- 5.2 Testing Strategy.....
- 5.3 Test Plan and Acceptance Criteria.....
- 5.4 Test Data and Environment.....
- 5.5 Test Cases and Recorded Results.....
- 5.6 Chapter Summary and Evaluation.....

### 6 System Deployment

- 6.1 Environment and Deployment Preparation.....
- 6.2 Frontend Ingress and API Addressing.....

|                                                    |  |
|----------------------------------------------------|--|
| 6.3 Backup Risk Management and Release Checks..... |  |
| 6.4 Handover and Chapter Evaluation.....           |  |
| <b>7 Discussions and Conclusion.....</b>           |  |
| 7.1 Project Summary                                |  |
| 7.2 Achievements.....                              |  |
| 7.3 Contributions.....                             |  |
| 7.4 Limitations and Future Improvements.....       |  |
| 7.5 Issues and Solutions.....                      |  |
| 7.6 Conclusion.....                                |  |
| <b>References</b>                                  |  |
| <b>Appendices</b>                                  |  |

# Chapter 1

## **Introduction**

# 1 Introduction

This report documents the backend system and web-based administration client of Campus Navigator. It explains how navigation events become operational information, how current user positions reach the browser and how the prototype can be reproduced locally. The mobile application supplies navigation data and shared map resources, but its sensor fusion and route-guidance implementation are outside this individual report's focus.

## 1.1 Project Objectives

The primary objective is to provide a usable administration view of the indoor navigation application. An operator should be able to identify popular destinations and observe named active sessions without interpreting internal node identifiers or reading service logs.

The backend objective is to reuse the existing presence and analytics services through explicit read interfaces. The client objective is to present only two principal pages: a dashboard and a live map. The verification objective is to exercise the complete path with 100 synthetic sessions and demonstrate that changing the visible marker limit does not change which sessions the backend accepts. A further objective is to document a reproducible Google Cloud deployment path without presenting a planned deployment as a completed release.

## 1.2 Project Background

A mobile navigation screen answers an individual question: where should this user go next? A management screen answers different questions: which destinations are frequently selected, how many journeys have started and which sessions are currently reporting movement? These questions require both historical events and recent presence, whose lifetimes and query patterns differ.

The project already contains Go services for presence ingestion, trajectory processing and analytics. Building the administration client around these services avoids introducing a second ingestion pipeline. However, integration requires more than drawing points: the browser must use the same coordinate origin as the mobile map, preserve display names and request an administrative snapshot that is not restricted to a mobile representative sample.

## 1.3 Advantages and Contributions

The contribution is a connected management workflow rather than a separate source of simulated frontend data. Names originate in sessions, positions originate in backend updates and summary values originate in stored journey events. Readable room labels make the dashboard interpretable to an operator who does not know the graph schema.

The implementation also separates display policy from data availability. Selecting ten markers simplifies a crowded screen while retaining the full eligible snapshot. Shared map metadata provides a repeatable coordinate transformation and reduces the risk of adjusting offsets by eye. These choices make the demonstration easier to explain and support future changes without changing the mobile positioning contract.

## **1.4 Project Plan**

Work follows an incremental sequence: inspect the service contracts, define the two administration pages, connect real read endpoints, run the local simulator, correct visible integration defects and prepare deployment instructions. Each stage has a concrete completion condition. For example, the live-map stage is complete only when backend-reported actors have visible names and align with the same floor surface as the application.

The remaining release stage is a controlled deployment on a Google Compute Engine VM. It requires a selected project, a configured VM, DNS ownership and a public-access decision. Those external prerequisites are not treated as completed by the existence of a deployment script.

## **1.5 Project Team and Organisation**

Pui Hock Yang, registration number 24WMR01790, prepares this Software Engineering report under the supervision of Prof. Ts. Dr. Tew Yiqi. The documented contribution concerns backend integration, the administration website, local simulation and deployment preparation.

The wider repository includes the mobile navigation client, floor resources and shared contracts. These components are integration dependencies. This report does not attribute every component in the shared repository to this individual contribution, and it does not infer additional team members or responsibilities that have not been confirmed.

## **1.6 Chapter Summary and Evaluation**

The scope is intentionally narrow: historical navigation summaries and a named live map. Service-health dashboards remain an operator concern outside the administration client. Administrator authentication is deferred for the synthetic demonstration, which means that the prototype must not be exposed with real-person names and positions without a separate access-control decision.

## Chapter 2

# Literature Review

## 2 Literature Review

This chapter reviews the technical approaches relevant to a navigation administration system. The comparison uses primary protocol and product documentation alongside the requirements of this implementation. It is a technology review, not a claim that a controlled user study or a comprehensive survey of commercial indoor-navigation products has been conducted.

### 2.1 Live Communication Approaches

WebSocket provides a bidirectional connection after an opening handshake (Fette & Melnikov, 2011). It fits the mobile presence path because sessions repeatedly submit movement updates. In this project, the simulator follows that same path through authenticated WebSocket messages.

The browser administration client uses periodic HTTP snapshots instead. It waits for a request to finish and schedules the next refresh roughly one second later. This reduces overlapping requests and keeps the first implementation understandable, but it introduces a refresh delay and repeated payload transfer. Therefore, live means frequently refreshed presence rather than an instantaneous or guaranteed real-time control channel.

### 2.2 Transient Presence and Event Processing

Redis consumer groups support processing stream entries with delivery tracking and acknowledgements (Redis, n.d.). The project uses streams to decouple ingestion from ClickHouse insertion. Recent presence is held separately in active-session indexes and session payloads so the map can query current state without scanning historical events.

Acknowledging a stream entry after a successful database insert reduces the chance of losing an event between delivery and insertion. It does not establish exactly-once processing, and retries can repeat work. It also does not protect an in-memory Redis deployment from losing unprocessed entries when the server restarts. The persistence configuration therefore matters as much as the stream API.

### 2.3 Historical Aggregation and Data Meaning

ClickHouse supports analytical storage and aggregation. Its `ReplacingMergeTree` documentation explains that replacement occurs during merges and is not an immediate guarantee that duplicate rows disappear from every query (ClickHouse, n.d.). This is relevant when workers retry inserts or when several lifecycle records describe one journey.

The dashboard must count the intended business event rather than raw records. Its query groups by journey identity before deriving totals, preventing route changes from being counted as additional navigation starts. A start count is still not a unique-person count: one session can begin several journeys. This distinction is part of the interface's meaning, not merely a database detail.

## 2.4 Deployment and Transport Options

Docker Compose describes an application through services, networks and volumes (Docker, n.d.). It is a practical fit for a demonstration that combines Go services, Redis, ClickHouse and a web server on one machine. A single VM also keeps private service discovery straightforward, though it creates a shared failure point.

A reserved Compute Engine external IP provides an address that can be associated with a DNS record (Google Cloud, n.d.). DuckDNS offers a free subdomain service (Duck DNS, n.d.), and Caddy automates certificate management for suitable public hostnames (Caddy, n.d.). These facilities support a free-name deployment, but they do not make VM resources, storage or external IP usage free. Public HTTPS and private container-network HTTP have different trust boundaries.

## 2.5 Comparison and Project Justification

A static website alone cannot execute the current frontend's server-side API proxy handlers. Hosting the existing client as a server-backed container preserves same-origin requests and runtime backend configuration. A direct browser-to-backend design is possible, but it would require a deliberate public API boundary and cross-origin configuration rather than exposing Docker service names.

A generic infrastructure dashboard can explain service availability, but it does not by itself provide room-name destination rankings or application-specific map coordinates. A dedicated administration client is justified by those domain requirements. The initial version favours a small interface and a reused backend over additional health panels or an unrelated reporting platform.

## 2.6 Chapter Summary and Evaluation

The reviewed approaches support a separation between ingestion, recent presence, historical aggregation and presentation. Their limitations guide the design: snapshot polling has delay, stream delivery is not automatic end-to-end durability, and DNS plus TLS does not supply administrator authorisation. These limitations remain explicit in the implementation and deployment plan.

# Chapter 3

## **Methodology and Requirements Analysis**

## 3 Methodology and Requirements Analysis

The methodology converts the requested administration experience into testable behaviour. Requirements were refined through project discussions, repository and API inspection, and iterative prototype feedback. Existing mobile-project documents inform chapter organisation but are not evidence of a separate backend survey or experiment.

### 3.1 Development Methodology

An iterative and incremental approach was used because the interface requirements became clearer after the first prototype. Early requests included broad analytics and monitoring ideas. Feedback narrowed the product to a navigation dashboard and a map resembling the application's two-dimensional floor view.

Each iteration connected one visible behaviour to its underlying data path. The map-origin correction, permanent name labels and complete administrative snapshot were handled as separate integration concerns. This approach allowed the demonstration to reveal semantic errors that isolated page-rendering tests would not identify.

### 3.2 Requirements Gathering

Discussion established the need for readable destination names, current user movement and display limits of 10, 20, 30 or all users. Repository inspection identified existing Go endpoints, Redis presence storage, ClickHouse events and map resources. Prototype review exposed the coordinate offset and the absence of visible names.

The local simulation requirement provided a repeatable way to assess integration without collecting real students' locations. One hundred synthetic sessions must send information through the backend; the browser must not create one hundred decorative points independently. The earlier mobile-document poll is not reused as proof of these administration requirements.

### 3.3 Stakeholders and Use Cases

The operator opens the dashboard, selects today or this week, and reads journey totals, outcomes and popular destinations. If no data is available or a request fails, the page must communicate that state rather than substitute fabricated activity.

On the live page, the operator inspects the shared floor map, chooses a marker limit, pans or zooms and selects a user for detail. The mobile client or simulator is the reporting actor: it establishes a session and sends movement updates. The backend validates and stores those updates. The website is a read client, not a controller that changes a user's navigation route.

### 3.4 Functional and Nonfunctional Requirements

FR1 requires day and week summaries with destination names. FR2 requires one marker per eligible active session and a visible display name. FR3 requires the 10, 20, 30 and All options to limit rendering only. FR4 requires map coordinates to use the same surface origin as the application. FR5 requires backend errors to remain distinguishable from empty results. FR6 requires reproducible local simulation and deployment instructions.

The nonfunctional requirements are data consistency, readable presentation, bounded refresh behaviour and clear operational boundaries. Historical statistics must not be relabelled as live occupancy. Backend endpoints must remain private in the proposed frontend-only ingress. No measured service-level latency, maximum capacity or positioning-accuracy guarantee is specified because the available verification does not establish those properties.

Administrator login, individual route control, service-health panels, a dedicated heatmap page and production access auditing are excluded from the current interface. Infrastructure monitoring remains separate. Deferring login is acceptable only for the agreed synthetic demonstration, not as a production privacy control.

### 3.5 Chapter Summary and Evaluation

The requirements define observable behaviour that can be checked against the backend, rather than visual appearance alone. They also prevent misleading interpretations: one marker denotes an active session, one navigation start denotes a journey, and a display limit is not an ingestion limit. Chapter 5 relates these requirements to the available verification evidence.

# Chapter 4

## **System Design**

## 4 System Design

The design has two read paths with a common ingestion source. Current positions flow through the Presence Gateway and Redis to the live map. Historical events flow through Redis Streams and the Trajectory Worker to ClickHouse, then through the Analytics API to the dashboard. The administration server exposes same-origin proxies to the browser.

### 4.1 System Context

The mobile application and the simulator both report graph-relative positions. The Presence Gateway owns session handling and current presence. It also emits trajectory and journey lifecycle events. The worker consumes those events asynchronously, while the analytics service answers historical queries without taking over session ingestion.

This separation prevents dashboard queries from becoming the source of live user state. It also allows the administration interface to remain read-only. The server-side web layer forwards requests to the appropriate Go service and maps identifiers to human-readable labels using the shared campus catalog.

Table 4.1 summarises the responsibility and data boundary of each main component.

Table 4.1 Responsibilities of the main components

| Component         | Responsibility                    | Data boundary              |
|-------------------|-----------------------------------|----------------------------|
| Presence Gateway  | Sessions and live snapshots       | Current eligible actors    |
| Trajectory Worker | Stream consumption and insertion  | Events in transit          |
| Analytics API     | Period summaries and rankings     | Historical journey records |
| Admin web         | Two pages and same-origin proxies | Read-only presentation     |

### 4.2 User Interface Design

The dashboard groups its information around a period selector, headline journey values, daily activity and destination rankings. Place names such as TA257 and Toilet 1 are displayed instead of graph node identifiers. A rank describes destination selection frequency within the chosen period and threshold, not a claim about the number of distinct people in a room.

The live page gives the map the main visual area. Name labels remain visible alongside markers, and the operator can reduce clutter with 10, 20, 30 or All. The application map image and the web map image share the same floor surface. Error and empty states remain visible without adding infrastructure status panels.

### 4.3 Data Design

A live actor includes a stable response identifier, `display_name`, graph position, sequence and update time. The position refers to a source node, destination node and edge progress. Display names are labels, not secure proof of identity, and names need not be unique. Rendering keys therefore use actor identity rather than the name alone.

Redis holds floor activity indexes and current presence payloads. The active sorted set supports a last-seen cutoff, while payload checks verify floor and building membership. ClickHouse holds trajectory and journey records for time-window aggregation. The immutable map bundle and generated campus catalog supply graph coordinates and readable destination labels.

### 4.4 Dashboard Semantics

The dashboard uses Malaysia calendar boundaries, UTC+8, and a Monday-based week. It first resolves journey records by identity and then aggregates starts and outcomes. Daily totals and the outcome categories should reconcile with the selected period's total.

Destinations with fewer than five starts are suppressed from the ranking. Suppression reduces low-count disclosure but does not anonymise the separate live map. An in-progress journey among the selected starts is not necessarily the same as a currently connected session. Labels and explanations must preserve these differences.

### 4.5 Security and Privacy Design

Mobile sessions use the backend's session and token mechanism, while the administration demonstration intentionally has no administrator login. These are separate authentication boundaries. A valid mobile session does not imply that a web viewer has been authorised to see everyone's positions.

The proposed ingress exposes public HTTPS only at the reverse proxy. Redis, ClickHouse and the Go service ports remain on the private Docker network. Runtime configuration contains internal service addresses, not browser-facing Docker URLs. Synthetic names and positions should be used until access control, consent, retention and audit requirements have been addressed.

### 4.6 Presence and Analytics Processes

For presence, the client establishes a named session and sends movement messages. The gateway validates and updates recent state. The administrative snapshot reads all eligible active floor members, fetches their payloads and excludes stale or mismatched records. The browser then applies its selected display limit.

For analytics, the gateway emits events into streams. The worker batches and inserts records into ClickHouse, then acknowledges processed entries. The Analytics API queries the stored records and returns structured totals and rankings. Because these stages are asynchronous, a newly reported journey may appear in the historical dashboard later than its actor appears on the live map.

## 4.7 Software Architecture and Interfaces

The three Go services have separate responsibilities and container boundaries. The administration client uses React and TypeScript with a server-backed Vnext build. Browser requests go to `/api/live` and `/api/dashboard`; their handlers call the Presence Gateway and Analytics API respectively.

The live backend read interface is `GET /v1/live/floors/{building_id}/{floor_id}`. The dashboard read interface is `GET /v1/analytics/dashboard` with `map_id` and `period` parameters. The legacy representatives response field still carries the live actor array, but the administrative path uses `SnapshotAll`. Its field name must not be mistaken for a ten-person storage cap.

## 4.8 Coordinate and Display Algorithms

The floor image is a cropped surface of an infinite Tiled map. For this surface, the origin is  $(-256, 256)$  and the image measures 1536 by 2048 pixels. A world point maps to image coordinates by subtracting the origin:  $\text{imageX} = \text{worldX} + 256$  and  $\text{imageY} = \text{worldY} - 256$ . The implementation derives the bounds from visible tile chunks rather than relying on a manually tuned screen offset.

For endpoints  $A$  and  $B$  and progress  $p$ , the displayed world point is  $A + \text{clamp}(p, 0, 1) \times (B - A)$ . The origin transformation is applied after interpolation. Zoom and pan then transform the image and overlay together. Sorting actors consistently before slicing the visible list makes the 10, 20 and 30 options predictable; All bypasses that slice but still respects backend eligibility.

## 4.9 Chapter Summary and Evaluation

The design preserves a single position contract across clients and separates current sessions from journey history. The key correctness boundaries are snapshot completeness, coordinate origin, event identity and metric meaning. A shared VM simplifies the demonstration, but neither that topology nor the current unauthenticated client is presented as a production availability or privacy design.

## Chapter 5

# Implementation and Testing

## 5 Implementation and Testing

This chapter records implementation details and the verification available from the local demonstration. It distinguishes completed synthetic checks from proposed field and cloud checks. Results are limited to the recorded local integration checks and are not a production benchmark.

### 5.1 Implementation Structure

The administration pages are implemented in admin-web, with separate dashboard and map components. API handlers forward to the real backend and do not silently fall back to fabricated users. A campus-generation script synchronises place labels and map metadata from the application resources. The coordinate helpers expose `surfaceFromTiled`, `worldToImage` and `positionOnImage` so their behaviour can be tested independently.

The gateway's administrative reader uses `SnapshotAll` instead of the representative mobile snapshot. It reads the floor's eligible active members and pipelines payload retrieval. The simulator supplies display names such as Walker 001, which pass through session storage and the live response. The web marker renders `display_name` visibly, rather than requiring a tooltip to reveal it.

### 5.2 Testing Strategy

Verification combines focused automated checks and an end-to-end Docker demonstration. Coordinate tests check the shared origin and interpolation behaviour. Page tests check the rendered administration routes. Backend tests cover service behaviour within their respective Go modules. These tests address different defects and are not interchangeable with visual inspection.

The integration verifier reads the real `/api/live` endpoint twice, checks unique actors and display names, and requires movement for at least 90% of the expected sessions. It also reads the weekly dashboard, reconciles outcomes and daily totals, checks destination labels and verifies rejection of an invalid period. It does not measure browser frame rate or a production percentile latency.

### 5.3 Test Plan and Acceptance Criteria

The live-map acceptance case starts 100 synthetic sessions, checks that the backend returns all eligible actors, and switches between 10, 20, 30 and All in the browser. Expected marker counts are 10, 20, 30 and 100 when all sessions remain active. Every visible marker must retain its display name. The API population must not decrease merely because the operator selects ten markers.

The alignment case compares graph endpoints and interpolated points with the shared floor surface. The dashboard case checks named destination rankings and arithmetic reconciliation for

the selected period. Failure cases must preserve an error or empty state rather than introduce fake successful data. Cloud acceptance separately requires DNS, a valid certificate and public proxy checks; it is not covered by local acceptance.

## 5.4 Test Data and Environment

The local Docker composition contains Redis, ClickHouse, the Presence Gateway, the Trajectory Worker, the Analytics API, the simulator and the administration web container. The demonstration address is `http://localhost:3100`. The simulator establishes 100 named sessions and sends graph-constrained movement through the actual backend, with journeys continuing between destinations.

This is synthetic integration data. The workload does not represent 100 recruited participants, measured pedestrian behaviour or a certified maximum capacity. Repeated simulation also changes journey totals over time, so this report does not freeze a changing dashboard number as a permanent experimental result. The commands needed to reproduce the demonstration are provided in Appendix A.

## 5.5 Test Cases and Recorded Results

The earlier local demonstration verified 100 unique named sessions and moving positions through the backend. Display selections of 10, 20, 30 and All produced the intended marker limits. The map-origin fix and name rendering addressed the two visible defects raised during prototype review.

The administration test suite contained five checks: three coordinate checks and two rendered-page checks. Deployment preparation additionally included five frontend-script tests and local configuration checks. A frontend container integration check exercised both pages, both proxy endpoints and the 100-user named snapshot. These results support the local demonstration only. No Google Cloud rollout, real-device positioning study, failure-recovery exercise or production load benchmark is reported as passed.

The Redis-list concern was resolved by distinguishing two readers of the same presence state. The mobile representative reader limits what it returns for its use case. The administrative reader requests all eligible active users. The website's marker selector is a subsequent client-side operation, so it is not evidence that Redis stores only ten or thirty users.

Table 5.1 separates the recorded local outcomes from the checks that remain outstanding.

Table 5.1 Verification scope and recorded outcomes

| Case            | Expected behaviour                 | Recorded outcome    |
|-----------------|------------------------------------|---------------------|
| Live population | 100 unique named sessions          | Verified locally    |
| Movement        | Positions change between reads     | Verified locally    |
| Display limits  | 10 / 20 / 30 / All markers         | Verified locally    |
| Coordinates     | Shared origin and interpolation    | Three checks passed |
| Web pages       | Both administration routes render  | Two checks passed   |
| Cloud release   | DNS, TLS and public APIs work      | Not yet executed    |
| Field accuracy  | Compare with physical ground truth | Not evaluated       |

## 5.6 Chapter Summary and Evaluation

The available checks establish functional integration for the synthetic workload and address the observed UI defects. They do not establish long-term durability, public deployment success or real-world position accuracy. A release decision should therefore require additional deployment and privacy checks rather than extrapolating from the local demonstration.

# Chapter 6

## **System Deployment**

## 6 System Deployment

The deployment target is a Google Compute Engine Ubuntu VM running the backend and administration frontend in Docker. This chapter describes the prepared workflow and its prerequisites. Google Cloud deployment has not yet been executed or verified for this report.

### 6.1 Environment and Deployment Preparation

Prepare an Ubuntu 24.04 VM, Docker Compose v2, a reviewed repository revision and persistent storage for ClickHouse. Reserve an external IPv4 address and configure only the required public ingress. A static address supports a stable DNS record, but its billing and lifecycle remain part of the cloud project (Google Cloud, n.d.). The free DNS name does not remove cloud-resource charges.

For the frontend-only workflow, register a DuckDNS name and deploy the backend first using the Google Cloud guide. Avoid starting a competing host reverse proxy on ports 80 and 443. The frontend script expects an existing backend network that resolves `presence-gateway` and `analytics-api`. It does not provision the complete backend or a DuckDNS account.

### 6.2 Frontend Ingress and API Addressing

Run `deploy/gcp/deploy-frontend.sh` inside the VM with the registered domain and reserved public IP. It validates inputs, checks the backend network and host ports, builds the client, checks the backend endpoints, updates DNS and starts its separate frontend composition. Its hidden token prompt avoids placing the DuckDNS token in command-line arguments.

The browser opens `https://YOUR_NAME.duckdns.org` and requests `/api/live` and `/api/dashboard` on that same origin. Caddy forwards to `admin-web` over the private network; the web server calls `http://presence-gateway:8080` and `http://analytics-api:9092` internally. The existing frontend-only script does not automatically expose mobile `/v1` routes on the same domain. A shared mobile and web hostname would require a reviewed combined proxy configuration.

### 6.3 Backup Risk Management and Release Checks

ClickHouse storage must be backed up and its restore procedure verified before important data is collected. The supplied production Redis configuration is ephemeral, so restarting it may lose current sessions and queued events that have not reached ClickHouse. A persistent analytics volume cannot recover events that were never inserted. Production durability requires a deliberate Redis persistence and recovery design.

Release checks should verify DNS resolution, HTTPS certificate validity, HTTP-to-HTTPS redirection, both pages, both proxy endpoints and private database exposure. Confirm that no unrelated reverse proxy is replaced. Confirm that real-person tracking is unavailable to unauthorised viewers before replacing synthetic data. Caddy certificate automation simplifies TLS operations but does not supply that access-control policy (Caddy, n.d.).

## **6.4 Handover and Chapter Evaluation**

The operator guide should demonstrate period selection, place-name interpretation, map controls and the difference between active sessions and journey counts. The developer guide should explain runtime environment variables, service discovery and safe updates. Appendix A provides local operation commands, while Appendix B identifies the implementation files and cloud deployment entry points.

The prepared scripts reduce repeated manual steps, but final acceptance remains pending a real VM run. One-command deployment still relies on correct prerequisites, available ports, owned DNS and working backend grants. The handover must retain that distinction and record the eventual public address only after it is actually deployed.

## Chapter 7

# **Discussions and Conclusion**

## 7 Discussions and Conclusion

The completed prototype connects the backend's current and historical data to a focused administration interface. Evaluation is limited to the verified local synthetic demonstration and the inspected implementation. This chapter relates the result to the objectives and identifies the work needed before a public release.

### 7.1 Project Summary

Campus Navigator now has a backend-connected dashboard and named live map. Redis supplies recent eligible presence, while the worker and ClickHouse support historical navigation summaries. The web client reuses the application's floor resources and places backend positions in the correct image coordinate system.

### 7.2 Achievements

The local demonstration supports the two requested administration tasks. Operators can review navigation starts and popular destination names, then inspect user movement with selectable marker counts. One hundred synthetic sessions exercised the backend path. Coordinate tests and prototype feedback led to a shared-origin transformation, and session names are now visible on the map.

Deployment preparation produced a Google Cloud workflow and a frontend script that keeps public HTTPS separate from private backend HTTP. This is a preparation achievement, not evidence that the application has already been deployed publicly.

### 7.3 Contributions

The contribution lies in reconciling backend semantics with an operator-facing interface. It makes the distinction between mobile representative snapshots and complete administrative snapshots explicit. It also connects human-readable destination and session labels to technical identifiers without treating labels as authentication.

The repeatable simulator and read-only verifier provide a practical demonstration method. They allow changes to be assessed against actual service responses and arithmetic consistency, rather than relying only on a populated-looking interface.

### 7.4 Limitations and Future Improvements

The current administration view has no login, the deployment uses a single VM and the live page transfers snapshots by polling. Real-person privacy, disaster recovery, high availability and

production capacity remain unresolved. Simulated movement does not validate the mobile sensor estimate against physical ground truth.

Future work should add administrator authorisation and audit trails before using real identities, strengthen event durability, measure end-to-end latency under controlled load and validate Google Cloud release and restore procedures. A streaming web channel or marker clustering could be considered only after measuring the present polling and rendering costs. Heatmaps remain a possible later feature, not a delivered third page.

## 7.5 Issues and Solutions

The visible map offset came from treating graph world coordinates as pixels relative to the cropped PNG. Subtracting the shared surface origin corrected the coordinate frame. The missing-name concern required checking the whole session-to-marker chain and making the existing display name permanently visible. The apparent user cap required an administrative all-user reader and a separate rendering limit.

A deployment ambiguity was resolved by retaining browser-facing HTTPS while allowing HTTP only between containers on the VM. This avoids asking an HTTPS page to call a public HTTP API. The remaining mobile public endpoint is a separate ingress decision rather than an implied consequence of deploying the web client.

## 7.6 Conclusion

The project meets the local prototype objective of making navigation activity understandable through a dashboard and a named live map. Its strongest result is an integrated, reproducible data path with explicit metric and coordinate semantics. The next milestone is a controlled Google Cloud deployment with appropriate access restrictions and operational verification, rather than adding unrelated administration panels.

## References

Caddy. (n.d.). Automatic HTTPS. Retrieved September 7, 2026, from <https://caddyserver.com/docs/automatic-https>

ClickHouse. (n.d.). ReplacingMergeTree table engine. Retrieved September 7, 2026, from <https://clickhouse.com/docs/reference/engines/table-engines/mergetree-family/replacingmergetree>

Docker. (n.d.). How Compose works. Retrieved September 7, 2026, from <https://docs.docker.com/compose/intro/compose-application-model/>

Duck DNS. (n.d.). About. Retrieved September 7, 2026, from <https://www.duckdns.org/about.jsp>

Fette, I., & Melnikov, A. (2011). The WebSocket protocol (RFC 6455). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6455>

Google Cloud. (n.d.). Configure static external IP addresses. Retrieved September 7, 2026, from <https://docs.cloud.google.com/compute/docs/ip-addresses/configure-static-external-ip-address>

Redis. (n.d.). XREADGROUP. Retrieved September 7, 2026, from <https://redis.io/docs/latest/commands/xreadgroup/>

# Appendices

## Appendix A Local Operator Guide

From the repository `tarumt-nav-app` directory, run `bash dev/admin-local.sh` up. With the stack running, execute `node dev/verify-admin-local.mjs`. Docker must be available, and the required local ports must be free. These commands start the demonstration and verify backend data; they are not cloud deployment commands.

Open `http://localhost:3100` for the dashboard and `http://localhost:3100/live` for the map. On the dashboard, choose today or this week and interpret totals as navigation starts, not distinct people. On the map, choose 10, 20, 30 or All, then pan or zoom as needed. A smaller display limit does not stop the other sessions from reporting.

The demonstration uses synthetic Walker names and has no administrator password. If the interface shows a backend error, inspect the local container logs and service configuration rather than treating the screen as an empty but healthy dataset. Stop the local stack using the documented `dev/admin-local.sh` command when it is no longer required.

## Appendix B Developer and Deployment Guide

The service entry points are `services/presence-gateway`, `services/trajectory-worker` and `services/analytics-api`. The administration code is in `admin-web`. Coordinate behaviour is isolated in `app/map-coordinates.mjs`, and `scripts/sync-campus.mjs` generates the shared catalog and surface metadata. Tests are located in `admin-web/tests` and the service test files.

The web server requires `PRESENCE_API_BASE_URL` and `ANALYTICS_API_BASE_URL`. In Docker these resolve to the private service addresses described in Chapter 6. Do not put database credentials or DNS tokens in browser code. Do not replace the real proxy handlers with static sample responses for a deployment demonstration.

For Google Cloud, use `docs/operations/google-cloud-deployment.md` and `deploy/Frontend-GCP.md`. The frontend entry point is `deploy/gcp/deploy-frontend.sh`. Its `--plan` option previews the intended operation. The actual run needs an already deployed backend, an owned DuckDNS name, the reserved IP and the token supplied securely at the prompt.

## Appendix C Evidence and Scope Notes

The outline follows `fyp_docs_only/modify`. Implementation evidence comes from the service source, administration code, Compose definitions and verification scripts. Chapter 5 records earlier local synthetic checks, not a new benchmark. No real-participant study, cloud address, physical accuracy result or maximum capacity is asserted.